# 4 Experimente und Ergebnisse

Dieses Kapitel berichtet über die Experimente auf dem GazeCapture-Datensatz [1] und auf den eigenen Aufnahmesitzungen. Zuerst werden das Modell, die Daten und das Evaluationsprotokoll beschrieben (Abschnitt 4.1). Danach werden die Trainings-Hyperparameter (Abschnitt 4.2), die räumliche Verteilung des Vorhersagefehlers und die Verlustfunktionen (Abschnitt 4.3) sowie die Personalisierungssitzungen (Abschnitte 4.4 bis 4.6) analysiert. Abschnitt 4.7 diskutiert, was diese Ergebnisse für die weitere Arbeit bedeuten.

Alle Werte wurden mit den Experimentprotokollen abgeglichen. Bei dieser Prüfung wurde ein Fehler im Auswertungscode gefunden und korrigiert (Abschnitt 4.1.3). Alle Modellfehler in diesem Kapitel werden mit der korrigierten Metrik angegeben.

## 4.1 Versuchsaufbau

### 4.1.1 Modell

- **Backbone:** ResNet-18 [2], vortrainiert auf ImageNet (`ResNet18_Weights.DEFAULT`).
- **Regressionskopf:** vier vollständig verbundene Schichten $512 \to 512 \to 256 \to 128 \to 2$ mit ReLU, Dropout $p = 0{,}2$ nach der zweiten Schicht und einer Ausgabe-Aktivierung (Standard: Sigmoid).
- **Optimierer:** AdamW, Weight Decay $10^{-5}$.
- **Standard-Verlust:** L1-Verlust (`nn.L1Loss`), siehe Abschnitt 4.3.3.
- **Early Stopping:** Geduld (Patience) von drei Epochen, höchstens 500 Epochen.

### 4.1.2 Daten und Splits

GazeCapture enthält in der hier verwendeten Version 347 085 gelabelte Frames von 262 Personen. Es werden zwei Splits verwendet:

- `norm_subject`: personendisjunkter Split (keine Person kommt sowohl in den Trainings- als auch in den Testdaten vor);
- `norm_random`: Split auf Frame-Ebene, dieselbe Person kann in Trainings- und Testdaten vorkommen.

Für die Hyperparameterstudie werden 1000 Trainings- und 200 Testframes *zufällig* aus den Split-Dateien gezogen (fester Seed 0, `-sample-seed 0`). Für `norm_subject` enthält die Trainingsteilmenge 179 Personen und die Testteilmenge 45 Personen ohne Überschneidung. (Frühere Läufe verwendeten die *ersten* Zeilen der sortierten Split-Dateien, d. h. nur fünf Trainingspersonen und eine Testperson; diese Läufe werden für die Tabellen nicht mehr verwendet.)

### 4.1.3 Evaluationsmetrik

Alle Blickziele sind auf das Einheitsquadrat $[0,1]^2$ normiert (Abschnitt 4.1.5). Der Fehler ist die Wurzel aus dem mittleren quadrierten *euklidischen* Abstand zwischen Vorhersage $\hat{\boldsymbol{p}}_i$ und Ziel $\boldsymbol{p}_i$, angegeben in Prozent der Bildschirmdiagonale $\sqrt{2}$:

$$E_{\text{diag}} \;=\; 100 \cdot \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|_2^2}}{\sqrt{2}} \quad [\%]. \tag{4.1}$$

Ein Wert von 0 % ist eine perfekte Vorhersage, 100 % ein Fehler so groß wie die gesamte Bildschirmdiagonale.

### 4.1.4 Protokoll

Die Hyperparameterstudie wird mit dem Befehl

```
python exp.py -experiment all -sample-seed 0 -val-size 200
```

ausgeführt. Jeweils eine Achse (zum Beispiel die Batchgröße) wird verändert, während alle anderen Werte auf der Standardkonfiguration bleiben (Tabelle 4.1). Jede Konfiguration wird fünfmal trainiert. Das Early Stopping verwendet eine personendisjunkte Validierungsmenge von 200 Frames, und angegeben wird der *Test*fehler in der Epoche mit dem niedrigsten Validierungsfehler; die Testmenge wird also nicht für die Modellauswahl verwendet. Die Tabellen zeigen den mittleren Testfehler, das zweiseitige 98 %-Konfidenzintervall (KI) des Mittelwerts (Student-$t$, vier Freiheitsgrade) und die mittlere Laufzeit (siehe unten).

**Was die „Laufzeit“ misst.** Die Spalte *Laufzeit* ist die tatsächlich verstrichene Zeit (Wall-Clock) *eines vollständigen Trainingslaufs*, gemittelt über die fünf Läufe. Ein Lauf beginnt mit der Auswertung des untrainierten Modells (Epoche 0) und endet, wenn das Early Stopping das Training beendet; er umfasst alle Trainingsepochen und die Auswertung auf den Trainings-, Validierungs- und Testdaten nach jeder Epoche. Sie ist *nicht* die Zeit pro Epoche und enthält *nicht* das Laden der Bilder oder die Berechnung der zwischengespeicherten Backbone-Merkmale, die einmal vor dem Lauf erfolgt. Da das Early Stopping die Läufe

nach unterschiedlich vielen Epochen beendet, spiegelt die Zeit auch wider, wie lange sich eine Konfiguration noch verbessert.

**Tabelle 4.1:** Standardkonfiguration von `exp.py`. Jede Variation ändert genau einen dieser Werte.

| Parameter | Wert | Parameter | Wert |
|---|---|---|---|
| Split | `norm_subject` | Trainierbare Schichten | nur FC |
| Trainings-/Testgröße | 1000 / 200[a] | Ausgabe-Aktivierung | Sigmoid |
| Batchgröße | 32 | Verlust | L1 (`standard`) |
| Lernrate | $10^{-4}$ | Max. Epochen / Patience | 500 / 3 |
| Weight Decay | $10^{-5}$ | Validierungsmenge | 200 Frames[b] |
| Wiederholungen | 5 | | |

[a] zufällig gezogen, Seed 0. [b] von einer Person, die weder in der Trainings- noch in der Testteilmenge vorkommt.

**Lesehilfe für die Tabellen.** In jeder Hyperparametertabelle ist die gewählte Option mit † markiert. Die Spalte „$p$ vs. †“ gibt den $p$-Wert eines zweiseitigen Mann-Whitney-$U$-Tests zwischen den fünf Läufen dieser Option und den fünf Läufen der gewählten Option an. Ein Rangtest wird verwendet, weil fünf Läufe zu wenige sind, um die Normalverteilung zu prüfen, und weil mehrere Optionen einzelne Ausreißerläufe enthalten (z. B. ein ReLU-Lauf mit 46,98 %). Bei $5 + 5$ Läufen ist der kleinstmögliche $p$-Wert 0,008; er bedeutet, dass *jeder* Lauf der einen Option besser ist als *jeder* Lauf der anderen. Ein Mittelwert ist nur dann **fett** gedruckt, wenn die gewählte Option *signifikant besser* ist als jede andere Option derselben Tabelle ($p < 0{,}05$ und niedrigerer Mittelwert).

Drei Eigenschaften dieses Protokolls sind zu beachten:

1. **Gleiche Daten in allen Wiederholungen.** Der Seed legt die Trainings-, Test- und Validierungsteilmengen fest, sodass sich die fünf Wiederholungen nur in Gewichtsinitialisierung, Datenreihenfolge und Dropout unterscheiden. Das KI misst daher die Zufälligkeit des Trainings, nicht die Variation zwischen verschiedenen Datenteilmengen.
2. **Kleine Validierungsmenge.** Die 200 Validierungsframes stammen von einer einzigen Person. Das Early Stopping ist daher verrauscht, die Testmenge wird aber nie für die Modellauswahl verwendet.
3. **Trainingszeit.** Bei Läufen mit „nur FC“ wird der Kopf auf zwischengespeicherten ResNet-Merkmalen trainiert (512 Zahlen pro Bild statt des Bildes). Ein Lauf dauert daher nur etwa eine Sekunde, und die Unterschiede zwischen diesen Zeiten sind nicht relevant. Die Zeiten von FC + Layer 4 und vollem ResNet enthalten den Vorwärts- und Rückwärtsdurchlauf durch den Backbone in jeder Epoche und sind daher viel länger; sie sind mit den Zeiten von „nur FC“ nicht direkt vergleichbar.

### 4.1.5 Normierung der Zielwerte

Die Roh-Blickziele von GazeCapture sind in Bildschirmpunkten des Aufnahmegeräts angegeben, gemessen von der oberen linken Bildschirmecke. Ihr Wertebereich hängt daher von der Bildschirmgröße und der Ausrichtung des Geräts ab: Die 347 085 Frames der 262 Personen stammen von zehn Bildschirmformaten, von $320 \times 480$ bis $1024 \times 768$ Punkten, aufgenommen im Hoch- und im Querformat. Die Ziele werden daher mit der Bildschirmbreite $W$ und -höhe $H$ des jeweiligen Frames normiert (`main.py`; Werte außerhalb von [0,1] werden abgeschnitten, was 0,006 % der Frames betrifft):

$$x_{\text{norm}} = \frac{x}{W}, \qquad y_{\text{norm}} = \frac{y}{H}. \tag{4.2}$$

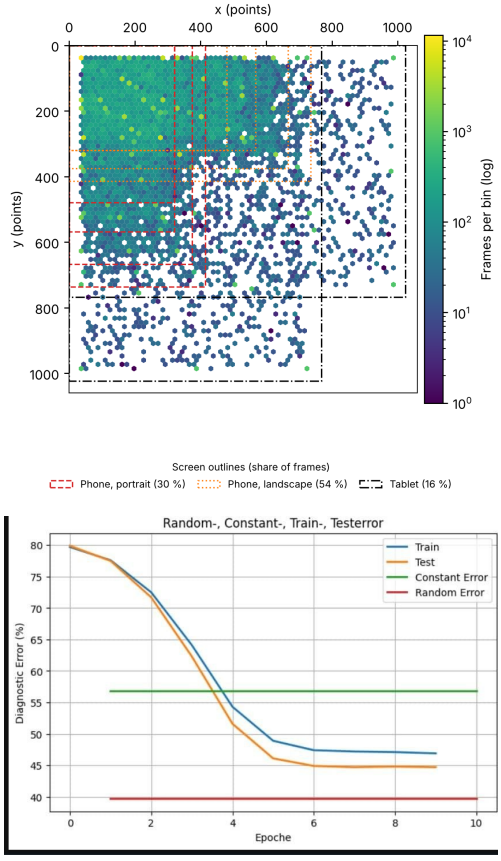
Abbildung 4.1 zeigt die Wirkung. Ohne Normierung konzentrieren sich die Ziele der vielen Smartphone-Frames im oberen linken Bereich, und dieselbe Koordinate bedeutet auf jedem Gerät eine andere Bildschirmposition (Abbildung 4.1a); die Fehlerwerte sind daher zwischen Geräten nicht vergleichbar (linke Spalte). Nach der Normierung liegen alle Ziele in $[0,1]^2$, und Gleichung (4.1) ist wohldefiniert (rechte Spalte). Alle weiteren Experimente verwenden normierte Ziele.

**Warum beginnt die Kurve bei der Konstant- und nicht bei der Zufalls-Baseline?** In Abbildung 4.1b beginnen die Kurven in Epoche 0, also *vor* dem ersten Trainingsschritt, bei 26,77 %. Man könnte erwarten, dass sich das untrainierte Modell wie die Zufalls-Baseline (etwa 40 %) verhält. Diese Erwartung trifft aus folgendem Grund nicht zu.

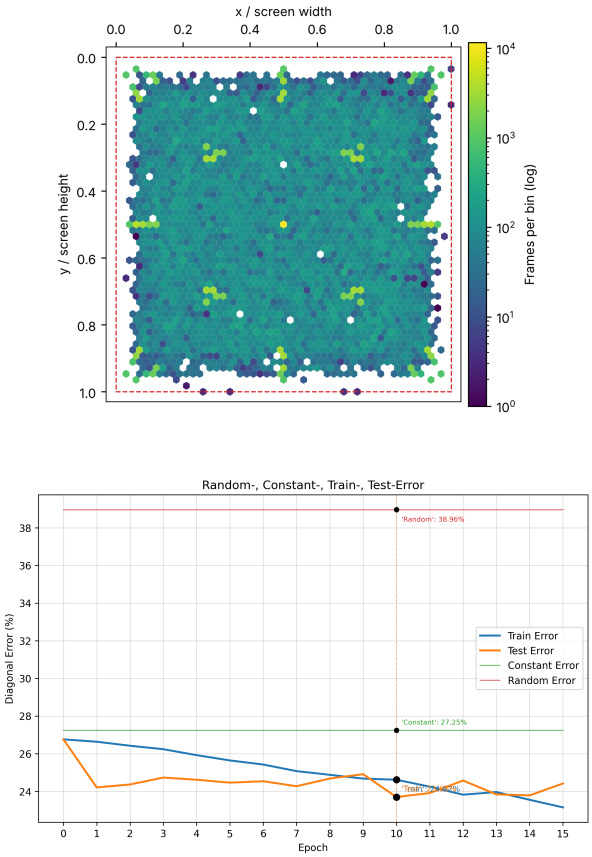
Die Zufalls-Baseline rät für jedes Bild einen *anderen*, gleichverteilten Punkt über den ganzen Bildschirm. Ein untrainiertes Netz tut das nicht. Seine Gewichte sind mit kleinen Zufallswerten initialisiert, daher erzeugt die letzte lineare Schicht für jedes Bild Werte nahe 0, und die Sigmoid bildet sie auf $\sigma(0) = 0{,}5$ ab. Das untrainierte Modell sagt daher für alle Bilder fast *denselben* Punkt vorher, die Bildschirmmitte (0,5; 0,5). Genau das tut die Konstant-Baseline. Die richtige Erwartung lautet daher:

- Epoche 0 $\approx$ Konstant-Baseline *derselben* Testdaten („immer in die Mitte schauen“);
- das Training muss die Kurve *unter* diesen Wert bringen; erst dann hat das Modell etwas aus den Bildern gelernt;
- die Zufalls-Baseline ist eine obere Referenz für „reines Raten“; ein Modell sollte nie darüber liegen.

Die gemessenen Werte bestätigen das: Auf den 200 Testbildern dieses Laufs beträgt die Konstant-Baseline 26,76 % (Trainingsmittelwert) bzw. 26,71 % (Bildschirmmitte), was dem Startwert von 26,77 % entspricht. Die waagerechte Linie in der alten Abbildung (27,25 %) wurde auf dem *gesamten* Datensatz berechnet und liegt daher etwas höher; dadurch schien die Kurve „unterhalb der Baseline“ zu beginnen. Der Plot-Code wurde korrigiert: Die




**(a)** Roh-Ziele in Bildschirmpunkten (Dichte aller Frames, logarithmische Farbskala); gestrichelt: Umrisse der Bildschirmformate. Unten: Lernkurve eines frühen Laufs auf Rohkoordinaten; die Werte sind keine Prozente einer gemeinsamen Bildschirmdiagonale und mit den anderen Abbildungen nicht vergleichbar.




**(b)** Normierte Ziele (gleiche Darstellung; gestrichelt: Einheitsquadrat). Die hellen Zellen sind Kalibrierpunkte, die auf allen Geräten an denselben relativen Positionen gezeigt werden. Unten: Lernkurve auf `norm_random` (1000/200) mit der Metrik aus Gleichung (4.1). Epoche 0 ist das untrainierte Modell. Die waagerechten Linien sind Baselines des *gesamten* Datensatzes; siehe Text.

**Abbildung 4.1:** Roh-Blickziele im Vergleich zu normierten Blickzielen.

Baselines werden jetzt auf derselben Trainings-/Testteilmenge wie die Kurven berechnet (`metrics.subset_baselines`; `Compare_notebook.ipynb`, Abschnitt 4, der diese Abbildung auch neu erzeugt).

### 4.1.6 Baselines

Es werden zwei Referenzschätzer verwendet. Die *Konstant-Baseline* sagt für jedes Testbild den mittleren Zielwert der Trainingsdaten vorher. Die *Zufalls-Baseline* zieht Vorhersagen gleichverteilt aus $[0,1]^2$. Ein Modell ist nur dann nützlich, wenn es deutlich besser ist als die Konstant-Baseline.

**Tabelle 4.2:** Baseline-Fehler $E_{\text{diag}}$ in %. Oben: gesamter Datensatz (`exp.py`, Bootstrap mit 30 Stichproben zu je 2000 Frames, Mittelwert und 98 %-KI). Unten: dieselben zufälligen Trainings-/Testteilmengen (Seed 0), die für das Modelltraining verwendet werden (kein KI, eine feste Teilmenge).

| Daten | Konstant-Baseline | Zufalls-Baseline |
|---|---|---|
| Gesamter Datensatz (347 085 Frames) | 27,28 [27,15; 27,40] | 39,82 [39,60; 40,04] |
| `norm_subject`, 1000/200 | 28,46 | 40,36 |
| `norm_subject`, 10 000/2000 | 27,62 | 39,96 |
| `norm_random`, 1000/200 | 27,66 | 39,83 |

Die Baselines des früheren Entwurfs (30,99 % und 42,70 %) stammten aus einem älteren Notebook-Lauf, der nicht Teil der Protokolle ist; sie sind durch die protokollierten Werte in Tabelle 4.2 ersetzt. Der untere Teil der Tabelle ist die richtige Referenz für alle Modellergebnisse in Abschnitt 4.2.

## 4.2 Trainingskonfiguration

Alle Ergebnisse dieses Abschnitts stammen aus einem Lauf (`exp_results_all.json`; `exp.py`, korrigierte Metrik, zufällige Teilmengen mit Seed 0, validierungsbasiertes Early Stopping, fünf Wiederholungen pro Konfiguration). Die Konstant-Baseline der Standardkonfiguration auf derselben Teilmenge beträgt 28,46 %.

### 4.2.1 Batchgröße

Tabelle 4.3 zeigt, dass die drei Batchgrößen denselben Testfehler ergeben: Die Mittelwerte unterscheiden sich um höchstens 0,05 Prozentpunkte, die Konfidenzintervalle überlappen, und der Rangtest findet keinen Unterschied ($p \geq 0{,}69$). Die Trainingszeiten liegen unter

**Tabelle 4.3:** Batchgröße. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines vollständigen Trainingslaufs (Kopf auf zwischengespeicherten Merkmalen, siehe Abschnitt 4.1.4).

| Batchgröße | Mittl. Testfehler (%) | 98 %-KI (%) | $p$ vs. † | Laufzeit (min) |
|---|---|---|---|---|
| 32† | 28,39 | [28,21; 28,58] | — | 0,019 |
| 64 | 28,35 | [28,29; 28,41] | 0,69 | 0,012 |
| 128 | 28,38 | [28,30; 28,47] | 0,84 | 0,009 |

einer Sekunde pro Lauf, weil nur der kleine Kopf auf zwischengespeicherten Merkmalen trainiert wird; die Zeitunterschiede sind daher nicht relevant.

Eine Batchgröße von 32 wird als Standard beibehalten. Das Experiment zeigt keinen Nachteil dieses Werts, und für eine kleine Batchgröße im Personalisierungssetting sprechen zwei praktische Argumente: (i) Eine persönliche Sitzung enthält nur einige hundert Trainingsbilder, sodass eine kleine Batchgröße mehr Parameterupdates pro Epoche ergibt, und (ii) dem Gradientenrauschen kleiner Batches wird üblicherweise eine leicht regularisierende Wirkung zugeschrieben. Die Daten dieses Experiments *belegen* jedoch keinen Vorteil von 32 gegenüber 64 oder 128.

### 4.2.2 Lernrate

**Tabelle 4.4:** Lernrate. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines vollständigen Trainingslaufs.

| Lernrate | Mittl. Testfehler (%) | 98 %-KI (%) | $p$ vs. † | Laufzeit (min) |
|---|---|---|---|---|
| $10^{-3}$ | 28,43 | [28,33; 28,54] | 0,15 | 0,018 |
| $10^{-4}$† | 28,37 | [28,32; 28,42] | — | 0,020 |
| $10^{-5}$ | 28,33 | [28,28; 28,38] | 0,056 | 0,022 |
| $10^{-6}$ | 28,37 | [28,21; 28,52] | 0,42 | 0,055 |

Tabelle 4.4 zeigt in der Variation mit fünf Läufen keinen signifikanten Unterschied zwischen den vier Lernraten: Alle Mittelwerte liegen zwischen 28,33 % und 28,43 %, die Konfidenzintervalle überlappen, und kein Rangtest ist signifikant ($p \geq 0{,}056$). Der Endfehler allein kann die Wahl daher nicht begründen. Die Wahl von $\eta = 10^{-4}$ beruht auf dem *Konvergenzverhalten* in Abbildung 4.2:

- $\boldsymbol{\eta = 10^{-3}}$: Die Schritte sind zu groß. Nach einem schnellen ersten Abfall schwankt der Testfehler stark und steigt wieder an (Spitzen von etwa 27,1 % und 28,4 %); der Lauf konvergiert nicht zu einem stabilen Minimum.

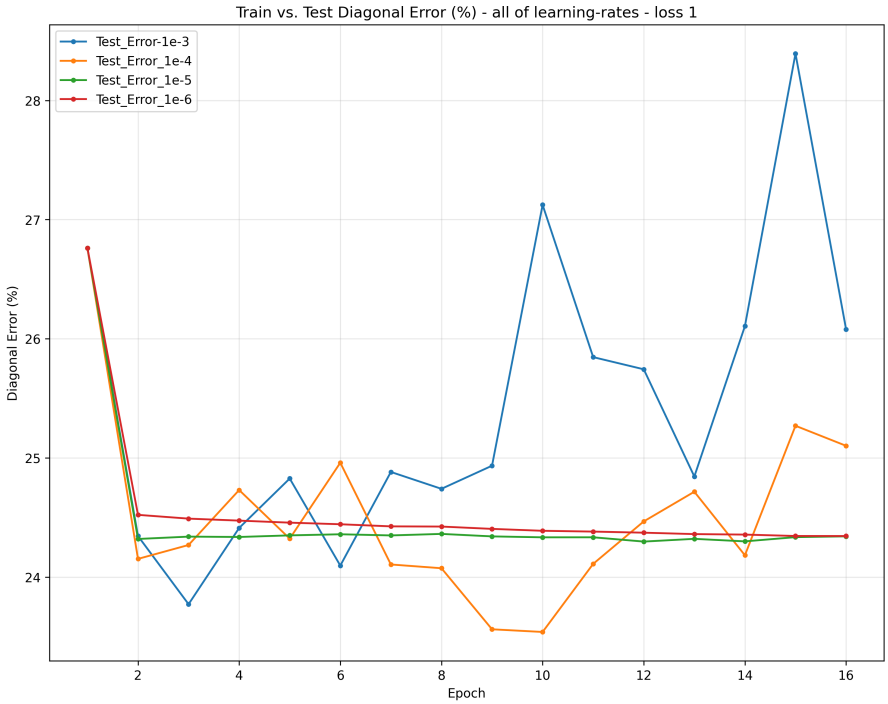



**Abbildung 4.2:** Testfehlerkurven einzelner Trainingsläufe mit vier Lernraten (`norm_random`, 1000/200, 15 Epochen ohne Early Stopping, Metrik aus Gleichung (4.1)). Der erste Punkt (Epoche 1 auf der Achse) ist das untrainierte Modell.

- $\boldsymbol{\eta = 10^{-4}}$: erreicht den niedrigsten Testfehler aller Kurven (etwa 23,6 % um Epoche 10). Die Kurve schwankt noch, aber viel weniger als bei $10^{-3}$, und das Early Stopping kann den besten Zustand behalten.
- $\boldsymbol{\eta = 10^{-5}}$ **und** $\boldsymbol{10^{-6}}$: Die Kurven sind sehr glatt und konvergieren monoton, bleiben aber auf einem Plateau von etwa 24,3 %. Die Schrittweite ist zu klein, um diesen Bereich innerhalb des Trainingsbudgets zu verlassen; das Ergebnis ist stabil, aber deutlich nicht das beste erreichbare.

Die Trainingszeit wird hier *nicht* als Kriterium verwendet: Sie liegt für alle Lernraten unter einer Sekunde (zwischengespeicherte Merkmale), und die Entscheidung hängt nur vom Konvergenzverhalten und vom erreichbaren Fehler ab. Zu beachten ist, dass Abbildung 4.2 einzelne Läufe zeigt; die Variation mit fünf Läufen in Tabelle 4.4 bestätigt keinen signifikanten Unterschied.

### 4.2.3 Trainierbare Schichten

Es werden drei Fine-Tuning-Tiefen verglichen (Tabelle 4.5): nur der Regressionskopf (nur FC), der Kopf plus die letzte Residualstufe (`layer4`) und das gesamte Netz.

**Tabelle 4.5:** Trainierbare Parameter der drei Fine-Tuning-Konfigurationen (ResNet-18 mit Regressionskopf, insgesamt 11 603 650 Parameter).

| | Nur FC | FC + Layer 4 | Volles ResNet |
|---|---|---|---|
| Trainierbare Parameter | 427 138 | 8 820 866 | 11 603 650 |
| Anteil an allen Parametern | 3,68 % | 76,02 % | 100,00 % |
| Backbone-Merkmale speicherbar | ja | nein | nein |

**Tabelle 4.6:** Trainierbare Schichten. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines vollständigen Trainingslaufs. Die Zeit für „nur FC“ enthält keinen Vorwärtsdurchlauf durch den Backbone (zwischengespeicherte Merkmale).

| Trainierbare Schichten | Mittl. Testfehler (%) | 98 %-KI (%) | $p$ vs. † | Laufzeit (min) |
|---|---|---|---|---|
| Nur FC† | 28,36 | [28,23; 28,49] | — | 0,015 |
| FC + Layer 4 | 29,07 | [26,58; 31,55] | 0,22 | 9,160 |
| Volles ResNet | 29,39 | [26,99; 31,78] | 0,31 | 15,366 |

„Nur FC“ hat den niedrigsten mittleren Fehler und mit Abstand das schmalste Intervall (Tabelle 4.6). Das Trainieren weiterer Schichten hilft nicht; es macht das Ergebnis *weniger stabil*: Einer der fünf Läufe mit FC + Layer 4 (31,72 %) und zwei der fünf Läufe mit vollständigem Fine-Tuning (30,70 % und 31,18 %) enden deutlich über der Konstant-Baseline (28,46 %), während alle Läufe mit „nur FC“ zwischen 28,28 % und 28,47 % bleiben. Mit nur fünf Läufen sind die Unterschiede nicht signifikant ($p = 0{,}22$ und 0,31), daher ist der Wert für „nur FC“ nicht fett gedruckt. Die Trainingszeit wächst stark mit der Zahl der trainierbaren Parameter: 9,2 min pro Lauf für FC + Layer 4 und 15,4 min für das gesamte Netz.

„Nur FC“ wird aus drei Gründen für die Personalisierungsexperimente gewählt:

1. **Kein Genauigkeitsverlust, mehr Stabilität:** In diesem Experiment senkt das Trainieren weiterer Schichten den Fehler nicht, sondern erhöht die Streuung zwischen den Läufen.
2. **Robustheit bei wenigen Daten:** Eine persönliche Sitzung hat nur einige hundert Bilder. Millionen von Backbone-Parametern mit so wenigen Beispielen zu aktualisieren, birgt die Gefahr der Überanpassung und verzerrt die vortrainierten Merkmale; Kumar et al. [3] zeigen, dass vollständiges Fine-Tuning vortrainierte Repräsentationen beschädigen und schlechter generalisieren kann als das Training nur des Kopfes.
3. **Aufwand:** Nur 3,68 % der Parameter werden trainiert, und die Backbone-Merkmale können einmal berechnet und wiederverwendet werden. Das macht die Personalisierung schnell und speichersparend.

### 4.2.4 Ausgabe-Aktivierung

Die Ausgabe-Aktivierung wird nach der letzten linearen Schicht angewendet, die $(x,y)$ erzeugt. Sigmoid bildet die Ausgabe auf (0,1) ab, was dem normierten Koordinatenbereich entspricht. Die Gauß-Aktivierung $g(z) = \exp(-z^2/2\sigma^2)$ mit $\sigma = 1$ bildet ebenfalls auf (0,1] ab, ist aber nicht monoton. ReLU entfernt nur negative Werte, und „Keine“ lässt die lineare Ausgabe unverändert.

**Tabelle 4.7:** Ausgabe-Aktivierung. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines vollständigen Trainingslaufs.

| Aktivierung | Mittl. Testfehler (%) | 98 %-KI (%) | $p$ vs. † | Laufzeit (min) |
|---|---|---|---|---|
| Sigmoid† | **28,36** | [28,29; 28,43] | — | 0,022 |
| Gauß | 31,29 | [28,44; 34,14] | 0,008 | 0,019 |
| ReLU | 32,90 | [19,69; 46,10] | 0,008 | 0,028 |
| Keine | 29,39 | [28,42; 30,36] | 0,008 | 0,032 |

Sigmoid ist der einzige klare Gewinner der Hyperparameterstudie (Tabelle 4.7): Sie hat den niedrigsten mittleren Fehler und das schmalste Intervall, und *jeder* Sigmoid-Lauf ist besser als *jeder* Lauf der drei anderen Aktivierungen ($p = 0{,}008$ in allen drei Vergleichen). Der Wert ist daher fett gedruckt. Die anderen Aktivierungen sind auch weniger stabil: Ein ReLU-Lauf endet bei 46,98 % und damit schlechter als die Zufalls-Baseline. Ein plausibler Grund ist, dass ReLU-Ausgaben für alle Eingaben null werden können („tote“ Ausgaben), wovon sich der Gradient nicht erholt. Ohne begrenzende Aktivierung (Keine, ReLU) kann das Netz außerdem Punkte außerhalb des Bildschirms vorhersagen. Sigmoid wird gewählt, weil sie die Ausgabe auf den gültigen Bildschirmbereich begrenzt und den niedrigsten und stabilsten Fehler ergab.

### 4.2.5 Datensatzgröße

**Tabelle 4.8:** Datensatzgröße (Training/Test). Mittlerer Testfehler über fünf Läufe mit 98 %-KI, mittlere Dauer eines vollständigen Trainingslaufs und Konstant-Baseline der jeweiligen Testteilmenge.

| Training / Test | Mittl. Testfehler (%) | 98 %-KI (%) | $p$ vs. † | Laufzeit (min) | Konst.-Baseline (%) |
|---|---|---|---|---|---|
| 10 000 / 2000 | 27,64 | [27,55; 27,73] | 0,008 | 0,167 | 27,62 |
| 1000 / 200† | 28,39 | [28,33; 28,44] | — | 0,018 | 28,46 |

Die größere Konfiguration hat einen niedrigeren mittleren Fehler (Tabelle 4.8). Die beiden Konfigurationen werden jedoch auf *unterschiedlichen* Testmengen ausgewertet, und jeder Mittelwert ist praktisch gleich der Konstant-Baseline seiner eigenen Testmenge (27,64 % vs. 27,62 % und 28,39 % vs. 28,46 %). Der Unterschied von 0,75 Prozentpunkten wird daher

durch die unterschiedlichen Testdaten verursacht und nicht durch mehr Trainingsdaten, auch wenn der Rangtest signifikant ist ($p = 0{,}008$). Die kleine Konfiguration (1000/200) wird für die Variationen beibehalten, weil sie etwa zehnmal schneller ist und der Datenmenge einer Personalisierungssitzung entspricht.

### 4.2.6 Split-Typ

**Tabelle 4.9:** Split-Typ. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und Konstant-Baseline der jeweiligen Testteilmenge.

| Split | Mittl. Testfehler (%) | 98 %-KI (%) | $p$ vs. † | Konstant-Baseline (%) |
|---|---|---|---|---|
| `norm_random` | 27,72 | [27,64; 27,80] | 0,008 | 27,66 |
| `norm_subject`† | 28,39 | [28,26; 28,53] | — | 28,46 |

Der Zufallssplit ergibt den niedrigeren Fehler (Tabelle 4.9). Auch hier stammt der größte Teil des Unterschieds aus den Testteilmengen: Die Konstant-Baseline ist auf der `norm_random`-Teilmenge bereits um 0,80 Prozentpunkte niedriger, und beide Modelle liegen innerhalb von 0,07 Prozentpunkten ihrer eigenen Baseline. Zusätzlich erlaubt der Zufallssplit, dass dieselben Personen in Trainings- und Testdaten vorkommen, was die Aufgabe erleichtert und die Schätzung optimistisch macht. Der personendisjunkte Split `norm_subject` wird daher als Standard verwendet: Er ist der einzige, der die Generalisierung auf unbekannte Personen misst, also die Situation vor der Personalisierung.

## 4.3 Räumliche Fehleranalyse und Verlustfunktionen

Der mittlere Fehler aus Abschnitt 4.2 verbirgt, *wo* auf dem Bildschirm die Fehler auftreten. Dieser Abschnitt analysiert zuerst die räumliche Fehlerverteilung (Abschnitte 4.3.1 und 4.3.2). Das beobachtete Muster motivierte positionsgewichtete Verlustfunktionen (Abschnitt 4.3.3), die in Abschnitt 4.3.4 ausgewertet werden.

### 4.3.1 Bildschirmeinteilung

Der normierte Bildschirm $[0{,}1]^2$ wird durch ein $3 \times 3$-Referenzraster mit drei Regionen beschrieben:

$$\begin{aligned} \text{Mitte:} &\quad (0{,}50;\, 0{,}50), \\ \text{Ränder:} &\quad (0{,}10;\, 0{,}50),\ (0{,}50;\, 0{,}10),\ (0{,}90;\, 0{,}50),\ (0{,}50;\, 0{,}90), \\ \text{Ecken:} &\quad (0{,}10;\, 0{,}10),\ (0{,}10;\, 0{,}90),\ (0{,}90;\, 0{,}10),\ (0{,}90;\, 0{,}90). \end{aligned}$$

Ein Testbeispiel mit Ziel $\boldsymbol{p}$ gehört zum Rasterpunkt $\boldsymbol{g}$, wenn $\|\boldsymbol{p}-\boldsymbol{g}\|_2 \le r$ mit $r = 0{,}15$ gilt. Der Fehler eines Rasterpunkts ist der mittlere euklidische Fehler $\|\hat{\boldsymbol{p}}-\boldsymbol{p}\|_2$ seiner Beispiele (in normierten Einheiten, d. h. als Anteil der Bildschirmseite; die Diagonale ist $\sqrt{2} \approx 1{,}41$). Der Fehler einer Region ist der Mittelwert über alle Beispiele ihrer Rasterpunkte. Diese Definitionen werden in allen Abbildungen und Tabellen dieses Abschnitts verwendet.

### 4.3.2 Fehler-Heatmaps

Abbildung 4.3 verwendet die 200 Testvorhersagen des Standardmodells (`norm_subject`, L1-Verlust; Datei `results/heatmap/base_loss1.csv`). Für eine Cursorposition $\boldsymbol{c}$ (Mitte, Rand oder Ecke) wird der mittlere Fehler $\bar{e}(\boldsymbol{c})$ aller Beispiele innerhalb des Radius $r$ berechnet und als isotrope Gaußfunktion dargestellt:

$$H(x,y) = \bar{e}(\boldsymbol{c})\,\exp\!\left(-\frac{\|(x,y)-\boldsymbol{c}\|_2^2}{2\sigma^2}\right), \qquad \sigma = 0{,}20. \tag{4.3}$$

Die Farbe kodiert daher die *Größe* des lokalen mittleren Fehlers, und die Lage des hellen Flecks zeigt, *wo* er gemessen wurde. Alle drei Teilbilder verwenden dieselbe Farbskala (0 bis zum größten Wert), sodass die Farben direkt vergleichbar sind.

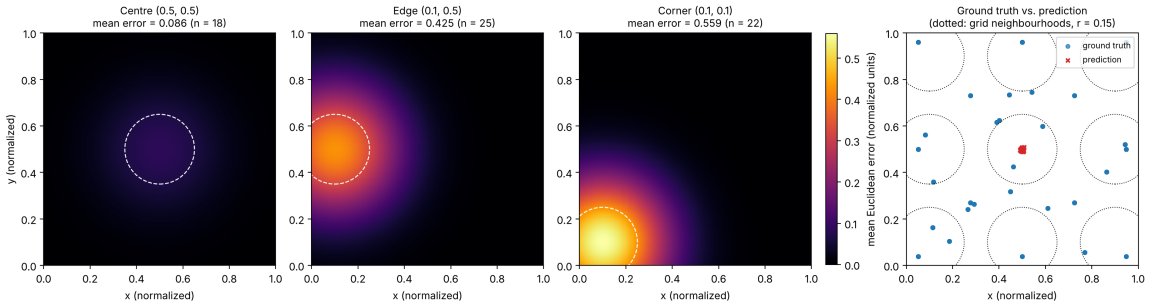



**Abbildung 4.3:** Räumlicher Fehler des Standardmodells (L1-Verlust, `norm_subject`, 200 Testbeispiele). Teilbilder 1–3: Gaußsche Fehler-Heatmaps ($r = 0{,}15$, $\sigma = 0{,}20$, gemeinsame Farbskala) in der Mitte (0,5; 0,5), am linken Rand (0,1; 0,5) und in der unteren linken Ecke (0,1; 0,1). Der Fehler wächst von der Mitte (0,086) über den Rand (0,425) zur Ecke (0,559). Teilbild 4: Ziele (blau) und Vorhersagen (rot); alle Vorhersagen liegen nahe der Bildschirmmitte.

Der Fehler nimmt von innen nach außen monoton zu: Der mittlere Fehler beträgt 0,086 in der Mittelregion ($n = 18$), 0,432 an den Rändern ($n = 62$) und 0,586 in den Ecken ($n = 42$). Die Randfehler sind etwa fünfmal und die Eckfehler etwa siebenmal so groß wie die Fehler in der Mitte.

Das vierte Teilbild erklärt dieses Muster. Alle Vorhersagen liegen in einem sehr kleinen Bereich um (0,5; 0,5): Ihre Standardabweichung beträgt in beiden Koordinaten etwa 0,004, während die Ziele eine Standardabweichung von etwa 0,28 haben. Das Modell sagt daher für

jedes Bild fast denselben Punkt vorher, und sein Fehler ist einfach der Abstand des Ziels zur Bildschirmmitte. Ein konstanter Mittelpunkt-Schätzer ergibt praktisch dieselben Werte (Mitte 0,085, Ränder 0,432, Ecken 0,587; insgesamt $E_{\text{diag}} = 28{,}45\,\%$ gegenüber $28{,}47\,\%$ für das Modell).

### 4.3.3 Verlustfunktionen

Die hohen Rand- und Eckfehler führten zu der Idee, diesen Regionen im Verlust mehr Gewicht zu geben. Sei $\hat{\boldsymbol{p}}_i = (\hat{x}_i, \hat{y}_i)$ die Vorhersage und $\boldsymbol{p}_i = (x_i, y_i)$ das Ziel des Beispiels $i$ in einem Batch der Größe $N$.

**L1-Verlust (`nn.L1Loss`, „standard“).**

$$\mathcal{L}_1 = \frac{1}{2N} \sum_{i=1}^{N} (|\hat{x}_i - x_i| + |\hat{y}_i - y_i|). \tag{4.4}$$

Der Gradient hat einen konstanten Betrag, unabhängig von der Fehlergröße. Der Verlust ist daher robust gegenüber Ausreißern (zum Beispiel falschen Labels während Zielübergängen), behandelt aber alle Bildschirmpositionen gleich.

**Smooth-L1-Verlust (`nn.SmoothL1Loss`, $\beta = 1$).** Für ein Residuum $u$ gilt:

$$\ell_\beta(u) = \begin{cases} \dfrac{u^2}{2\beta}, & |u| < \beta, \\ |u| - \dfrac{\beta}{2}, & \text{sonst.} \end{cases} \tag{4.5}$$

Er ist quadratisch für kleine und linear für große Residuen. Da alle Residuen in $[0{,}1]^2$ kleiner als $\beta = 1$ sind, verhält er sich hier wie ein skalierter quadratischer Fehler: Der Gradient wird klein, wenn der Fehler klein ist.

**Sensitiver L1-Verlust (positionsgewichtet, `sensitive`).** Jedes Beispiel wird mit seinem $L_1$-Abstand zur Bildschirmmitte gewichtet:

$$d_i = |x_i - 0{,}5| + |y_i - 0{,}5| \in [0{,}1], \qquad w_i = 1 + d_i \in [1{,}2], \tag{4.6}$$

$$\mathcal{L}_{\text{sens}} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \frac{|\hat{x}_i - x_i| + |\hat{y}_i - y_i|}{2}. \tag{4.7}$$

Ein Eckbeispiel hat das doppelte Gewicht eines Beispiels in der Mitte, ein Randbeispiel etwa das 1,4-fache. Das vergrößert den Gradienten für äußere Ziele und soll die Vorhersagen

von der Mitte wegziehen. (Der frühere Entwurf definierte das Gewicht als $w = 1/d$; das entspricht nicht der Implementierung in `model.py` und wäre in der Mitte unendlich.)

Der „verbesserte sensitive Verlust“ in Tabelle 4.11 war eine weitere Variante aus einem explorativen Notebook. Seine genaue Formel ist im aktuellen Code nicht enthalten; er wird daher nur der Vollständigkeit halber angegeben.

### 4.3.4 Ergebnisse des Verlustvergleichs

Tabelle 4.10 verbindet den globalen Vergleich aus der Variation mit fünf Läufen mit den räumlichen Fehlern der in Abschnitt 4.3.1 definierten Regionen. Sie ersetzt die beiden getrennten Tabellen des früheren Entwurfs (globale Verlusttabelle und $3 \times 3$-Rastertabelle).

**Tabelle 4.10:** Vergleich von L1- und sensitivem L1-Verlust. Links: mittlerer Testfehler $E_{\text{diag}}$ über fünf Läufe (`exp.py`, 98 %-KI) und mittlere Laufzeit. Rechts: mittlerer euklidischer Fehler pro Bildschirmregion (normierte Einheiten; je ein früherer Lauf, 200 Testbeispiele, $r = 0{,}15$; $n$ = Anzahl der Beispiele). Letzte Zeile: konstanter Mittelpunkt-Schätzer auf denselben 200 Testbeispielen.

| | Global (5 Läufe) | | | Räumlicher Fehler (1 Lauf) | | |
|---|---|---|---|---|---|---|
| Verlust | $E_{\text{diag}}$ (%) | 98 %-KI (%) | Laufzeit (min) | Mitte ($n = 18$) | Ränder ($n = 62$) | Ecken ($n = 42$) |
| L1 (standard)[†] | **28,32** | [28,23; 28,42] | 0,011 | 0,086 | 0,432 | 0,586 |
| Sensitiver L1 | 28,48 | [28,19; 28,78] | 0,021 | 0,090 | 0,431 | 0,585 |
| Konstante Mitte | 28,45[a] | — | — | 0,085 | 0,432 | 0,587 |

[a] Ausgewertet auf denselben 200 Testbeispielen wie die räumliche Analyse.

**Tabelle 4.11:** Hilfsexperiment: räumlicher Fehler von vier Verlustfunktionen nach 15 Epochen auf einer Trainingsmenge mit 52 Eckbeispielen (zwei Läufe, A und B, Werte wie protokolliert). Diese Zahlen stammen aus einem explorativen Notebook; die Berechnung der Regionswerte dieses Laufs ist im aktuellen Code nicht enthalten, daher sind sie mit Tabelle 4.10 nicht direkt vergleichbar.

| | Gesamt (%) | | Mitte (%) | | Ränder (%) | | Ecken (%) | |
|---|---|---|---|---|---|---|---|---|
| Verlust | A | B | A | B | A | B | A | B |
| L1 (`nn.L1Loss`) | 24,54 | 24,63 | 17,01 | 16,74 | 28,28 | 28,98 | 34,93 | 35,07 |
| Smooth L1 (`nn.SmoothL1Loss`) | 23,94 | 24,91 | 15,40 | 16,41 | 29,78 | 30,59 | 34,13 | 35,17 |
| Sensitiver L1 | 25,57 | 25,55 | 19,11 | 18,61 | 29,87 | 29,93 | 33,41 | 34,20 |
| Verbesserter sensitiver L1 | 25,53 | 27,75 | 19,20 | 21,86 | 29,17 | 35,03 | 33,77 | 31,58 |

**Beobachtungen.**

- Global ist L1 leicht, aber signifikant besser als der sensitive Verlust: 28,32 % gegenüber 28,48 % (Mann-Whitney $p = 0{,}032$, Tabelle 4.10); der L1-Wert ist daher fett gedruckt. Der sensitive Verlust streut außerdem stärker zwischen den Läufen. Beide Werte liegen nahe an der Konstant-Baseline (28,46 %).
- Räumlich verändert die Positionsgewichtung die Regionsfehler um weniger als 0,005 normierte Einheiten. Die Rand- und Eckfehler bleiben etwa fünf- bis siebenmal so hoch wie der Fehler in der Mitte.
- Beide Modelle haben dasselbe räumliche Profil wie ein konstanter Mittelpunkt-Schätzer. Die Gewichtung vergrößert den Gradienten für äußere Beispiele, das reicht aber nicht, damit das Modell die Mittelpunktlösung verlässt.
- Im Hilfsexperiment (Tabelle 4.11) senken die sensitiven Varianten den Eckfehler leicht (sensitiver L1 um 0,9–1,5, die verbesserte Variante um 1,2–3,5 Prozentpunkte), erhöhen aber den Fehler in der Mitte um 1,9–5,1 Prozentpunkte und den Gesamtfehler in allen Läufen. Die beiden Läufe unterscheiden sich deutlich, daher sind diese Tendenzen nicht verlässlich.

**Fazit.** Die räumliche Analyse hat gezeigt, *was* das Problem ist: Die Fehler konzentrieren sich an den Rändern und in den Ecken. Der positionsgewichtete Verlust war der Versuch, diese Fehler zu verringern. Er hat die Ecken nicht verbessert und war global sogar leicht, aber signifikant schlechter als L1 ($p = 0{,}032$); im Hilfsexperiment erhöhte er den Fehler in der Mitte. Der Standard-L1-Verlust wird daher beibehalten. Er ergab den niedrigeren Fehler, ist einfacher, hat keinen zusätzlichen Hyperparameter, und sein konstanter Gradientenbetrag macht ihn robust gegenüber Label-Rauschen in den persönlichen Aufnahmen. Die Analyse zeigt auch, *warum* die Gewichtung nicht helfen konnte: Die großen äußeren Fehler entstehen nicht durch eine falsche Gewichtung der Bildschirmregionen, sondern dadurch, dass das Modell die Bildschirmmitte kaum verlässt (Abschnitt 4.7).

## 4.4 Personalisierungssitzungen

### 4.4.1 Aufbau

In jeder persönlichen Sitzung wird der Kopf eines GazeCapture-Basismodells auf den Frames einer Aufnahme neu trainiert (nur FC, Sigmoid, L1, $\eta = 10^{-4}$, Weight Decay $10^{-5}$, Batchgröße 32, AdamW). Sofern nicht anders angegeben, wird jede Sitzung zufällig in 70 % Trainings-, 15 % Validierungs- und 15 % Testframes aufgeteilt (Seed 42). Das Early Stopping (Patience 3) verwendet den Validierungsfehler, und angegeben wird der Testfehler in der Epoche mit dem niedrigsten Validierungsfehler. Ist die beste Epoche 0, hat die Personalisierung den Validierungsfehler nicht verbessert, und der angegebene Testfehler ist der des unveränderten Basismodells.

Alle Ergebnisse dieses und der folgenden Abschnitte wurden mit dem Skript `personal_exp.py` (Notebook `personal_compare.ipynb`) und der korrigierten Metrik (Gleichung (4.1)) erzeugt. Alle Zufalls-Seeds sind fest, und der Backbone bleibt einschließlich seiner BatchNorm-Statistiken vollständig eingefroren.

**Merkmals-Cache.** Da der Backbone eingefroren ist, bildet er ein Bild in jeder Epoche auf dieselben 512 Merkmale ab. Diese Merkmale werden einmal pro Sitzung und Basismodell berechnet (Evaluationstransformation, ohne Augmentierung) und gespeichert; danach wird nur noch der Regressionskopf trainiert. Das verändert das trainierte Modell nicht, sorgt aber dafür, dass ein Lauf auf einer CPU nur Sekunden dauert. Der einzige Unterschied zu den früheren Notebook-Läufen ist, dass während der Personalisierung keine Bildaugmentierung angewendet wird.

**Basismodelle (E0).** Die beiden Basismodelle werden mit der in Abschnitt 4.2 gewählten Konfiguration (nur FC, Sigmoid, L1, $\eta = 10^{-4}$, Batchgröße 32, 1000/200 zufällige Frames, Seed 0, validierungsbasiertes Early Stopping) auf `norm_subject` und `norm_random` trainiert. Von fünf Seeds wird der Lauf mit dem niedrigsten *Validierungs*fehler behalten (`train_base.py`, `models/base/base_<split>.pt` mit einer JSON-Datei, die die Konfiguration und alle Seeds dokumentiert). Für `norm_subject` wurde Seed 0 gewählt (Validierung 27,37 %, Test 28,70 %; Konstant-Baseline derselben Testdaten 28,46 %), für `norm_random` Seed 3 (Validierung 26,74 %, Test 27,81 %; Konstant-Baseline 27,66 %). Die fünf Seeds unterscheiden sich um weniger als 0,4 Prozentpunkte, die Wahl des Seeds ist also unkritisch. Wie in Abschnitt 4.7 liegen beide Basismodelle auf dem Niveau der Konstant-Baseline.

Von den 27 aufgenommenen Sitzungen werden 18 verwendet: Gen 1 (01–04), Gen 2 (05–09) und Gen 4 (19–27). Gen 3 (10–18) wurde verworfen, weil sie keinen Lerneffekt zeigte. Tabelle 4.12 listet die Aufnahmebedingungen auf.

### 4.4.2 Ergebnisse aller Sitzungen (E1)

Tabelle 4.13 zeigt jede Sitzung mit beiden Basismodellen.

**Die Personalisierung wirkt.** Über alle 18 Sitzungen sinkt der Testfehler von 30,32 % (Konstant-Baseline) auf 19,79 % mit dem Basismodell `norm_subject` und auf 19,59 % mit dem Basismodell `norm_random`, eine Verringerung um 35 % bzw. 36 % (Wilcoxon-Vorzeichen-Rang-Test $p = 1{,}5 \cdot 10^{-5}$ für beide; gepaarter $t$-Test $p < 10^{-6}$). Die unveränderten Basismodelle sind nicht besser als die Konstant-Baseline (30,57 % und 30,51 %; Wilcoxon $p = 0{,}25$ und $p = 0{,}35$), der gesamte Gewinn stammt also aus den wenigen Minuten persönlicher Daten. Die beste Sitzung, S02 (Gen 1, heller Raum, ohne Brille), erreicht

**Tabelle 4.12:** Aufnahmebedingungen der 18 Sitzungen. Gen 1: kontinuierliche Zielbewegung ohne Sprünge; Gen 2: Ziel mit zufälligen Sprüngen; Gen 4: 5-s-Segmente (0,5 s WAIT an einem neuen Startpunkt, 4,5 s MOVE), 90 % der Startpunkte in den Bildschirmecken. Dauer und Betriebssystem wurden nur in Gen 4 variiert (Gen 1/2: Ubuntu, Dauer nicht protokolliert).

| Sitzungen | Generation | Brille | Licht / Hintergrund |
|---|---|---|---|
| 01, 03, 04 | Gen 1 | mit | 01, 04: hell; 03: dunkel |
| 02 | Gen 1 | ohne | hell |
| 05, 07, 08 | Gen 2 | mit | 05: hell; 07: dunkel; 08: sehr dunkel |
| 06, 09 | Gen 2 | ohne | 06: hell; 09: sehr dunkel |
| Gen 4 | (Dauer / BS) | | |
| 19, 20 | Gen 4 | mit | ruhig; 120 s / 90 s; Ubuntu |
| 21, 22 | Gen 4 | ohne | ruhig; 120 s / 90 s; Ubuntu |
| 23 / 24 | Gen 4 | mit / ohne | unruhig; 120 s; Ubuntu |
| 25 / 26 | Gen 4 | mit / ohne | unruhig; 120 s; Windows |
| 27 | Gen 4 | mit | unruhig; 90 s; Windows |

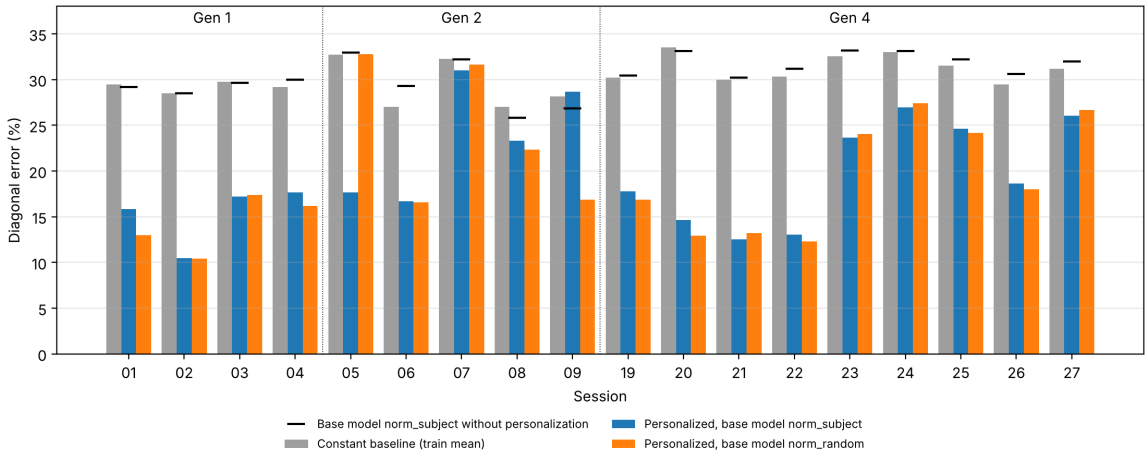



**Abbildung 4.4:** Testfehler der 18 Sitzungen nach der Personalisierung mit den beiden Basismodellen im Vergleich mit der Konstant-Baseline und mit dem unveränderten Basismodell (`norm_subject`) auf denselben Testframes (Daten aus Tabelle 4.13).

**Tabelle 4.13:** Personalisierung der 18 Sitzungen mit zwei Basismodellen (ursprüngliche Labels, zufälliger 70/15/15-Split, Seed 42). Basis: Testfehler des unveränderten Basismodells (Epoche 0); Val.: bester Validierungsfehler; Test: Testfehler in der besten Validierungsepoche; Ep.: beste Epoche; $t$: Dauer eines Laufs in Sekunden (Abschnitt 4.4.3); Konst.: Konstant-Baseline (Mittelwert der Trainingsziele, ausgewertet auf denselben Testframes). Fehler in % (Gleichung (4.1)). Gruppenzeilen: Mittelwert und 98 %-KI über die Sitzungen der Gruppe.

| | | Basismodell `norm_subject` | | | | | Basismodell `norm_random` | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Sitzung | Gen. | Basis | Val. | Test | Ep. | $t$ (s) | Basis | Val. | Test | Ep. | $t$ (s) | Konst. |
| 01 | 1 | 29,19 | 14,82 | 15,83 | 22 | 2,3 | 29,49 | 11,70 | 12,97 | 47 | 4,2 | 29,45 |
| 02 | 1 | 28,48 | 10,08 | 10,48 | 30 | 2,2 | 28,42 | 10,04 | 10,43 | 44 | 3,2 | 28,49 |
| 03 | 1 | 29,61 | 15,26 | 17,20 | 33 | 2,5 | 30,13 | 15,41 | 17,35 | 36 | 2,6 | 29,76 |
| 04 | 1 | 29,98 | 16,88 | 17,64 | 17 | 2,1 | 29,38 | 15,23 | 16,16 | 31 | 2,7 | 29,17 |
| 05 | 2 | 32,92 | 25,43 | 17,64 | 26 | 1,1 | 32,49 | 31,93 | 32,77 | 2 | 0,2 | 32,72 |
| 06 | 2 | 29,29 | 11,31 | 16,67 | 54 | 1,8 | 28,89 | 12,09 | 16,55 | 55 | 2,0 | 27,02 |
| 07 | 2 | 32,18 | 24,84 | 30,99 | 16 | 0,7 | 32,43 | 25,02 | 31,62 | 18 | 0,6 | 32,28 |
| 08 | 2 | 25,82 | 23,65 | 23,30 | 24 | 0,8 | 26,32 | 23,38 | 22,31 | 23 | 0,6 | 27,00 |
| 09 | 2 | 26,83 | 23,55 | 28,64 | 4 | 0,2 | 26,21 | 15,19 | 16,87 | 47 | 1,9 | 28,17 |
| 19 | 4 | 30,42 | 13,38 | 17,76 | 33 | 2,8 | 30,04 | 12,13 | 16,85 | 47 | 4,4 | 30,19 |
| 20 | 4 | 33,14 | 16,06 | 14,64 | 23 | 1,9 | 33,01 | 14,58 | 12,93 | 35 | 2,7 | 33,49 |
| 21 | 4 | 30,18 | 15,03 | 12,51 | 39 | 4,6 | 30,46 | 15,80 | 13,20 | 23 | 2,6 | 29,97 |
| 22 | 4 | 31,17 | 13,54 | 13,02 | 22 | 2,2 | 30,87 | 12,53 | 12,32 | 37 | 3,1 | 30,32 |
| 23 | 4 | 33,15 | 23,48 | 23,65 | 10 | 1,4 | 32,59 | 24,02 | 24,05 | 10 | 2,0 | 32,57 |
| 24 | 4 | 33,09 | 28,77 | 26,97 | 7 | 0,9 | 32,84 | 29,53 | 27,42 | 6 | 1,0 | 33,01 |
| 25 | 4 | 32,19 | 26,98 | 24,61 | 19 | 2,4 | 33,00 | 26,74 | 24,13 | 26 | 2,8 | 31,54 |
| 26 | 4 | 30,60 | 21,93 | 18,61 | 20 | 2,3 | 30,39 | 21,40 | 17,97 | 37 | 4,5 | 29,48 |
| 27 | 4 | 31,98 | 26,05 | 26,06 | 19 | 1,5 | 32,29 | 26,27 | 26,68 | 19 | 1,7 | 31,19 |
| Gen 1 (4) | | 29,32 | 14,26 | 15,29 | 25,5 | 2,3 | 29,35 | 13,10 | 14,23 | 39,5 | 3,2 | 29,22 |
| | | [27,86; 30,77] | [7,62; 20,89] | [7,80; 22,77] | | [1,8; 2,7] | [27,75; 30,96] | [7,07; 19,12] | [7,10; 21,35] | | [1,5; 4,8] | [27,99; 30,45] |
| Gen 2 (5) | | 29,41 | 21,76 | 23,45 | 24,8 | 0,9 | 29,27 | 21,52 | 24,02 | 29,0 | 1,0 | 29,44 |
| | | [24,14; 34,68] | [11,88; 31,63] | [12,74; 34,16] | | [0,0; 1,9][a] | [24,06; 34,47] | [8,20; 34,85] | [10,93; 37,12] | | [0,0; 2,5][a] | [24,68; 34,20] |
| Gen 4 (9) | | 31,77 | 20,58 | 19,76 | 21,3 | 2,2 | 31,72 | 20,33 | 19,51 | 26,7 | 2,8 | 31,31 |
| | | [30,60; 32,94] | [14,66; 26,50] | [14,26; 25,26] | | [1,2; 3,3] | [30,51; 32,93] | [13,88; 26,79] | [13,59; 25,42] | | [1,7; 3,9] | [29,91; 32,70] |
| **Alle (18)** | | **30,57** | **19,50** | **19,79** | **23,2** | **1,9** | **30,51** | **19,05** | **19,59** | **30,2** | **2,4** | **30,32** |
| | | [29,27; 31,87] | [15,87; 23,13] | [16,18; 23,40] | | [1,3; 2,5] | [29,21; 31,82] | [14,86; 23,25] | [15,47; 23,71] | | [1,6; 3,2] | [29,12; 31,53] |

[a] Untere Grenze des $t$-Intervalls unter null (wenige Sitzungen, große Streuung), bei 0 abgeschnitten.

10,5 %. Die neuen Läufe bestätigen die früheren Notebook-Läufe (Mittelwert 19,85 % über dieselben Sitzungen mit dem Basismodell `norm_subject`).

**Gruppen.** Die Verringerung gegenüber der Konstant-Baseline beträgt 48 % / 51 % in Gen 1, 20 % / 18 % in Gen 2 und 37 % / 38 % in Gen 4 (`norm_subject` / `norm_random`). In Gen 4 ist die Verbesserung signifikant (Wilcoxon $p = 0{,}004$); für Gen 1 ($n = 4$) und Gen 2 ($n = 5$) beträgt der kleinstmögliche $p$-Wert des Tests 0,125 bzw. 0,0625, sodass diese Gruppen für ein signifikantes Ergebnis zu klein sind, obwohl der Effekt in Gen 1 groß ist. Die Unterschiede zwischen den Generationen sind nicht signifikant (Kruskal-Wallis $p = 0{,}13$ und $p = 0{,}12$) und dürfen nicht allein als Wirkung des Aufnahmeprotokolls gelesen werden (Abschnitt 4.6).

**Wenn die Personalisierung scheitert.** In zwei Läufen ist das personalisierte Modell nicht besser als die Konstant-Baseline: S09 mit `norm_subject` (beste Epoche 4, 28,64 % gegenüber 28,17 %) und S05 mit `norm_random` (beste Epoche 2, 32,77 % gegenüber 32,72 %). In beiden Fällen hörte der Validierungsfehler sehr früh auf, sich zu verbessern. Mit dem jeweils anderen Basismodell erreichen dieselben Sitzungen 16,87 % (S09) und 17,64 % (S05). Das Scheitern ist daher keine Eigenschaft der Sitzung, sondern eines einzelnen Laufs: Mit nur 20–30 Validierungsframes (Gen 2) kann das Early Stopping das Training nach wenigen Epochen beenden. S07 (dunkler Raum) verbessert sich mit beiden Basismodellen nur wenig (31,0 % / 31,6 % gegenüber 32,3 %); gute Beleuchtung bleibt eine Voraussetzung für die Webcam-Personalisierung.

**Basismodell.** Beide Basismodelle ergeben dasselbe Ergebnis (19,79 % gegenüber 19,59 %; Wilcoxon $p = 0{,}20$). Das passt zum Hauptergebnis, dass das Basismodell vor allem eine Initialisierung liefert und die persönlichen Daten das Ergebnis bestimmen. Für die folgenden Experimente wird das Basismodell `norm_subject` verwendet, weil es ohne Informationen über die Testpersonen trainiert wurde.

**Validierungs- und Testfehler.** Im Mittel liegt der Validierungsfehler (19,50 %) nahe am Testfehler (19,79 %). Die größten Abweichungen (5–8 Prozentpunkte, S05–S07, S09) treten in den kurzen Gen-2-Aufnahmen auf, in denen die Validierungs- und Testmengen jeweils nur etwa 25 Frames enthalten.

### 4.4.3 Trainingszeit einer Sitzung

Die Zeit $t$ in Tabelle 4.13 ist die tatsächlich verstrichene Dauer *eines vollständigen Personalisierungslaufs* einer Sitzung auf den zwischengespeicherten Merkmalen: die Auswertung des unveränderten Modells (Epoche 0), alle Trainingsepochen bis zum Early Stopping und

die Auswertung der Trainings-, Validierungs- und Testdaten nach jeder Epoche. Sie ist weder eine Zeit pro Epoche noch ein Mittelwert über mehrere Läufe. Sie enthält nicht die einmalige Extraktion der Backbone-Merkmale der Sitzung, die alle Läufe derselben Sitzung und desselben Basismodells gemeinsam nutzen. Ein Lauf dauert auf einer CPU im Mittel 1,9 s (`norm_subject`) bzw. 2,4 s (`norm_random`); die Dauer wächst mit der Zahl der Frames und der Zahl der Epochen (kurze Gen-2-Aufnahmen: 0,9 s). Die früheren Notebook-Läufe, die jedes Bild in jeder Epoche durch den Backbone schickten, benötigten 1–26 Minuten pro Sitzung. Die „Laufzeit“ der Hyperparametertabellen (Abschnitt 4.1.4) ist dagegen der Mittelwert über fünf wiederholte Läufe einer GazeCapture-Konfiguration.

### 4.4.4 Label-Bereinigung (Gen 4, E1b)

In Gen 4 beginnt jedes Segment mit einer WAIT-Phase von 0,5 s, in der das Ziel an einer neuen Position erscheint und der Nutzer es erst finden muss. Diese Frames (10,0–10,5 % der Labels) können ein falsches Blicklabel haben. Sie sind exakt protokolliert, daher lassen sich bereinigte Labeldateien ohne die WAIT-Frames schreiben (`session_design.py -write-clean`).

Ein einfacher Vergleich ist nicht fair: Das bereinigte Modell wird ohne WAIT-Frames getestet, das ursprüngliche Modell dagegen auf allen Frames, einschließlich der schwer vorherzusagenden WAIT-Frames. Deshalb wird das ursprüngliche Modell zusätzlich nur auf den MOVE-Frames seines Testsplits ausgewertet (dasselbe Modell, WAIT-Frames aus der Testmenge entfernt; Tabelle 4.14).

**Tabelle 4.14:** Label-Bereinigung in Gen 4 (Basismodell `norm_subject`, zufälliger Split, Seed 42). Ursprünglich: auf allen Labels trainiert; Testfehler auf allen Testframes, nur auf den MOVE-Frames und nur auf den WAIT-Frames ($n_W$ = Anzahl der WAIT-Testframes). Bereinigt: ohne WAIT-Frames trainiert und getestet. $\Delta$ = Bereinigt − Ursprünglich (MOVE).

| | Ursprüngliche Labels | | | | Bereinigte Labels | | |
|---|---|---|---|---|---|---|---|
| Sitzung | Alle | MOVE | WAIT | $n_W$ | Test | Konst. | $\Delta$ |
| 19 | 17,76 | 13,28 | 41,12 | 8 | 13,06 | 29,07 | −0,22 |
| 20 | 14,64 | 12,24 | 28,38 | 8 | 12,77 | 30,71 | +0,53 |
| 21 | 12,51 | 9,77 | 27,59 | 10 | 9,79 | 29,23 | +0,01 |
| 22 | 13,02 | 10,21 | 27,64 | 8 | 9,96 | 31,71 | −0,25 |
| 23 | 23,65 | 20,95 | 41,84 | 10 | 17,18 | 30,92 | −3,77 |
| 24 | 26,97 | 24,96 | 41,93 | 10 | 23,99 | 31,64 | −0,97 |
| 25 | 24,61 | 21,63 | 44,36 | 10 | 17,94 | 30,49 | −3,70 |
| 26 | 18,61 | 14,34 | 41,72 | 10 | 14,58 | 30,20 | +0,24 |
| 27 | 26,06 | 23,10 | 44,80 | 8 | 23,69 | 30,59 | +0,60 |
| Mittel | 19,76 | 16,72 | 37,71 | 9,1 | 15,88 | 30,51 | −0,84 |

**Ergebnis.** Die WAIT-Frames werden sehr schlecht vorhergesagt (37,7 %, über der Konstant-Baseline): Der Blick ist noch nicht auf dem Ziel, ihre Labels sind also falsch. Werden sie nur aus der *Test*menge entfernt, sinkt der Fehler von 19,76 % auf 16,72 %. Werden sie auch aus den *Trainings*daten entfernt, sinkt er weiter auf 15,88 %, ein mittlerer Gewinn von 0,84 Prozentpunkten, der nicht signifikant ist (Wilcoxon $p = 0{,}43$). Der Gewinn stammt aus zwei Sitzungen mit unruhigem Hintergrund (S23, S25, etwa −3,7 Punkte); in den ruhigen Sitzungen liegt der Unterschied unter 0,6 Punkten. Fazit: Die WAIT-Frames sollten zumindest aus der Auswertung entfernt werden, weil sie den Labelfehler und nicht den Modellfehler messen; sie auch aus dem Training zu entfernen, kostet nichts und hilft in schwierigen Sitzungen, der Effekt ist aber klein.

**Variabilität von Lauf zu Lauf.** Ein früherer, ungültiger Vergleich bestand aus zwei Notebook-Läufen derselben Konfiguration auf identischen Daten (Gen 4, Zufalls-Seeds nicht fest). Sie unterschieden sich im Mittel um 1,9 und höchstens um 5,5 Prozentpunkte. Die neuen Läufe mit drei festen Seeds (Tabelle 4.15) zeigen eine Standardabweichung von bis zu 3,1 Prozentpunkten zwischen den Seeds. Unterschiede zwischen *einzelnen* Läufen in dieser Größenordnung sind daher nicht interpretierbar; das erklärt auch die oben genannten Fälle S05/S09.

### 4.4.5 Aufnahmebedingungen (Gen 4)

Gen 4 erlaubt Vergleiche, in denen sich nur ein Faktor ändert. Jeder Wert ist die Differenz zweier einzelner Sitzungen (Testfehler aus Tabelle 4.13, `norm_subject` / `norm_random`, Prozentpunkte; positiv = die erste Bedingung ist schlechter):

- **Brille** (mit − ohne): +5,2/ + 3,6 (S19–S21), +1,6/ + 0,6 (S20–S22), −3,3/ − 3,4 (S23–S24), +6,0/ + 6,2 (S25–S26);
- **Dauer** (90 s − 120 s): −3,1/ − 3,9 (S20–S19), +0,5/ − 0,9 (S22–S21);
- **Hintergrund** (unruhig − ruhig): +5,9/ + 7,2 (S23–S19), +14,5/ + 14,2 (S24–S21);
- **Betriebssystem** (Windows − Ubuntu): +1,0/ + 0,1 (S25–S23), −8,4/ − 9,4 (S26–S24).

Der einzige einheitliche und große Effekt ist der **Hintergrund**: Ein unruhiger Hintergrund erhöht den Fehler mit beiden Basismodellen um 6–15 Prozentpunkte. Eine Brille erhöht den Fehler tendenziell (drei von vier Paaren), die Aufnahmelänge (90 vs. 120 s) zeigt keinen klaren Effekt, und der Vergleich der Betriebssysteme ist widersprüchlich. Jede Bedingung ist durch einzelne Sitzungen vertreten, daher sind diese Werte beschreibend und nicht statistisch getestet.

## 4.5 Strategien zum Laden der Daten (E2)

Der zufällige Split aus Abschnitt 4.4 hat eine Schwäche: Benachbarte Frames eines Videos sind fast identisch (6 Labels pro Sekunde), sodass die Testmenge Beinahe-Duplikate von Trainingsframes enthält. Das ist eine Form von Datenleck (Data Leakage), die den zufälligen Split optimistisch macht. Der folgende Vergleich misst, wie groß dieser Effekt ist.

**Sitzungen.** Verwendet werden die Sitzungen 21 und 22. Sie werden nach einer festen Regel ausgewählt: Sie sind die einzigen zwei Sitzungen, die sich nur in einem Faktor unterscheiden (Dauer, 120 s vs. 90 s), und wurden ansonsten unter den besten Bedingungen aufgenommen (Gen 4, ohne Brille, ruhiger Hintergrund, Ubuntu, gleicher Tag). Das macht sie für den Transfer zwischen zwei Videos geeignet, und beide haben genug Frames (720 und 540) für zeitliche Blöcke.

**Strategien.**

**Zufällige Auswahl** (`-split random`): 70 % / 15 % / 15 % der Frames werden zufällig aus der Sitzung gezogen (Referenz; enthält Beinahe-Duplikate).

**Sequenzielle Auswahl** (`-split sequential`): Die Frames bleiben in Aufnahmereihenfolge; die ersten 70 % werden für das Training, die nächsten 15 % für die Validierung und die letzten 15 % für den Test verwendet. Eine Sekunde (6 Labels) am Anfang des Validierungs- und des Testblocks wird verworfen, damit keine benachbarten Frames auf beiden Seiten einer Blockgrenze liegen. Das entspricht dem realen Anwendungsfall: Der Nutzer kalibriert zuerst und verwendet den Tracker danach.

**Training mit zwei Videos** (`-split two_videos`): 80 % der Frames der ersten Sitzung werden für das Training und die übrigen 20 % (zufällig) für die Validierung verwendet; die zweite Sitzung dient *nur* dem Test (alle Frames). Beide Richtungen werden ausgeführt (21→22 und 22→21). Das misst, ob sich eine Kalibrierung auf eine neue Aufnahme derselben Person übertragen lässt.

**Fairer Vergleich.** Alle Strategien verwenden dasselbe Basismodell (`norm_subject`), dieselben Trainingseinstellungen, Early Stopping auf ihren eigenen Validierungsdaten und dieselben drei Seeds (42, 43, 44). Für jeden Lauf werden die Konstant-Baseline (Mittelwert der *Trainings*ziele, ausgewertet auf den Testframes dieses Laufs) und der Fehler des unveränderten Basismodells auf denselben Testframes angegeben, damit die unterschiedlichen Testmengen vergleichbar sind. Kein Testframe wird für das Training oder die Modellauswahl verwendet.

**Tabelle 4.15:** Strategien zum Laden der Daten (Sitzungen 21 und 22, Basismodell `norm_subject`, ursprüngliche Labels). Mittelwert über drei Seeds; SD: Standardabweichung des Testfehlers über die Seeds; Basis: unverändertes Basismodell; Red.: Verringerung des Testfehlers relativ zur Konstant-Baseline; $n$: Trainings- / Validierungs- / Testframes; $t$: Dauer eines Laufs (s).

| Strategie | Training → Test | Test (%) | SD | Val. (%) | Basis (%) | Konst. (%) | Red. (%) | $n$ | $t$ (s) |
|---|---|---|---|---|---|---|---|---|---|
| Zufällig | 21 → 21 | 14,40 | 3,13 | 14,15 | 31,52 | 31,25 | 53,9 | 503 / 108 / 109 | 3,9 |
| | 22 → 22 | 13,30 | 0,38 | 15,98 | 30,97 | 30,68 | 56,7 | 378 / 81 / 81 | 3,1 |
| Sequenziell | 21 → 21 | 14,47 | 1,22 | 19,09 | 27,62 | 27,43 | 47,2 | 503 / 102 / 103 | 2,0 |
| | 22 → 22 | 16,63 | 0,14 | 14,41 | 28,83 | 29,91 | 44,4 | 378 / 75 / 75 | 2,4 |
| Zwei Videos | 21 → 22 | 16,79 | 0,13 | 15,67 | 31,05 | 32,17 | 47,8 | 576 / 144 / 540 | 5,0 |
| | 22 → 21 | 17,70 | 0,42 | 14,33 | 32,03 | 33,13 | 46,6 | 432 / 108 / 720 | 3,2 |
| Zufällig (Mittel) | | 13,85 | | | 31,25 | 30,97 | 55,3 | | |
| Sequenziell (Mittel) | | 15,55 | | | 28,22 | 28,67 | 45,8 | | |
| Zwei Videos (Mittel) | | 17,24 | | | 31,54 | 32,65 | 47,2 | | |

**Ergebnis.** Die erwartete Reihenfolge wird bestätigt: zufällig (13,85 %) < sequenziell (15,55 %) < zwei Videos (17,24 %). Mit 6 Läufen pro Strategie (zwei Sitzungen × drei Seeds) ist nur der Unterschied zwischen sequenziellem Training und Training mit zwei Videos signifikant (Mann-Whitney U, $p = 0{,}009$); zufällig vs. sequenziell ($p = 0{,}09$) und zufällig vs. zwei Videos ($p = 0{,}06$) sind es nicht. Da sich die Konstant-Baselines zwischen den Testmengen unterscheiden, ist die relative Verringerung das fairere Maß: 55 % beim zufälligen Split, aber nur 46 %–47 % bei den realistischen Strategien. Der zufällige Split überschätzt den Nutzen der Personalisierung also um etwa 8–9 Prozentpunkte der Verringerung.

**Interpretation.**

- Der zufällige Split ist optimistisch, auf diesen beiden guten Sitzungen aber nur mäßig (1,7 Prozentpunkte). Bei schwierigen Sitzungen ist der Effekt viel größer (Abschnitt 4.6.2).
- Eine Kalibrierung überträgt sich auf eine zweite Aufnahme derselben Person unter denselben Bedingungen: Das Modell mit zwei Videos erreicht auf einem nie gesehenen Video 17 %–18 % bei 47 % Verringerung. Der Transfer kostet gegenüber dem zufälligen Split etwa 3 Prozentpunkte.
- Die Ergebnisse mit zwei Videos schwanken am wenigsten zwischen den Seeds (SD unter 0,5), da die Testmenge groß ist (540–720 Frames). Der zufällige Split hat die größte Streuung (SD 3,1 in S21), weil seine 81–109 Testframes vom Seed abhängen.
- Der Validierungsfehler des sequenziellen Splits sagt seinen Testfehler schlecht voraus (S21: 19,1 % Validierung vs. 14,5 % Test), weil Validierungs- und Testblock unterschiedliche Bildschirmbereiche abdecken.

Für den Anwendungsfall eines Eye-Trackers, der einmal kalibriert und danach benutzt wird, sind die Werte des sequenziellen Splits bzw. der zwei Videos die realistische Schätzung.

## 4.6 Gestaltung der Personalisierungssitzung

Die Gestaltung der Sitzung bestimmt, welche Bildschirmpositionen durch die Kalibrierdaten abgedeckt werden. Da die Fehler an den Rändern und in den Ecken am größten sind (Abschnitt 4.3), soll die Gestaltung diese Bereiche gut abdecken. Es wurden drei Varianten aufgenommen:

- **Standardvideo** (Gen 1, `base_Code.py`): Das Ziel bewegt sich kontinuierlich und ohne Sprünge über den Bildschirm;
- **Zufällige Sprungpunkte** (Gen 2): Das Ziel springt alle 1,4–2,7 s zu einem gleichverteilt gezogenen neuen Startpunkt;
- **Eckengewichtete Sprungpunkte** (Gen 4, `base_Code_rand.py`): 5-s-Segmente mit 0,5 s WAIT an einem neuen Startpunkt und 4,5 s MOVE; 90 % der Startpunkte liegen in den vier Eckbereichen (je 10 % × 10 % des Bildschirms).

### 4.6.1 Räumliche Abdeckung und Übergangsframes

Tabelle 4.16 wird direkt aus den Label- und Positionsdateien der 18 Sitzungen berechnet (`session_design.py`); ein Training ist nicht nötig. Die Regionsanteile verwenden die Bildschirmeinteilung aus Abschnitt 4.3.1 ($r = 0{,}15$). Übergangsframes sind Labels, die aufgenommen wurden, während sich der Blick noch zu einem neuen Ziel bewegt: in Gen 4 die protokollierte WAIT-Phase, in Gen 2 die Labels innerhalb von 0,5 s nach einem Sprung (geschätzt, keine Phasenprotokollierung), in Gen 1 keine.

**Tabelle 4.16:** Räumliche Abdeckung und Übergangsframes pro Gestaltung (Mittelwert über die Sitzungen jeder Generation). Zellen: Anteil der 100 Zellen eines 10 × 10-Bildschirmrasters, die mindestens ein Label enthalten; Abst.: mittlerer Abstand der Ziele von der Bildschirmmitte (normierte Einheiten).

| Gestaltung | Labels | Länge | Sprünge | Übergang | Anteil der Labels (%) | | | Zellen | Abst. |
|---|---|---|---|---|---|---|---|---|---|
| | | (s) | | (%) | Mitte | Ränder | Ecken | (%) | |
| Standard (Gen 1, 4 Sitz.) | 493 | 82 | 0 | 0 | 7,1 | 25,0 | 23,0 | 97 | 0,383 |
| Zufällige Sprünge (Gen 2, 5 Sitz.) | 179 | 30 | 16 | 28,5[a] | 6,3 | 23,0 | 26,3 | 57 | 0,384 |
| Eckengewichtet (Gen 4, 9 Sitz.) | 644 | 107 | 20 | 10,1 | 6,1 | 19,5 | 36,1 | 91 | 0,420 |

[a] Geschätzt: Labels innerhalb von 0,5 s nach einem Sprung.

#### Beobachtungen.

- **Die Eckengewichtung wirkt wie beabsichtigt:** Gen 4 hat 36 % seiner Labels in den Eckumgebungen, verglichen mit 23 % (Gen 1) und 26 % (Gen 2), und den größten mittleren Abstand von der Mitte.

- **Eine kontinuierliche Bewegung deckt den Bildschirm am gleichmäßigsten ab:** Gen 1 erreicht 97 % der Rasterzellen. Die kurzen Gen-2-Aufnahmen (30 s) decken nur 57 % ab.
- **Übergangsframes:** Mit Sprüngen alle 1,4–2,7 Sekunden und ohne Phasenprotokollierung fällt bis zu ein Drittel der Gen-2-Labels in einen Übergang; Gen 4 hat 10 %, die exakt protokolliert sind und entfernt werden können (Abschnitt 4.4.4).

### 4.6.2 Leistung der Personalisierung (E3)

Der zufällige Split begünstigt Gestaltungen mit langsamer, kontinuierlicher Bewegung, weil benachbarte, fast identische Frames in Trainings- und Testdaten landen. Deshalb wurden alle 18 Sitzungen zusätzlich mit dem sequenziellen Split ausgeführt (Abschnitt 4.5; Basismodell `norm_subject`, Seed 42). Tabelle 4.17 vergleicht beide Splits pro Gestaltung, und Tabelle 4.18 zeigt die Fehler pro Bildschirmregion, berechnet aus den gespeicherten Testvorhersagen.

**Tabelle 4.17:** Testfehler pro Gestaltung mit zufälligem und sequenziellem Split (Mittelwert über die Sitzungen; Basismodell `norm_subject`). Konst.: Konstant-Baseline der jeweiligen Testframes; Ep. 0: Anzahl der Sitzungen, in denen die beste Epoche 0 war (keine Verbesserung gegenüber dem Basismodell); $n_{\text{Test}}$: mittlere Anzahl der Testframes.

| | Zufälliger Split | | Sequenzieller Split | | | |
|---|---|---|---|---|---|---|
| Gestaltung | Test | Konst. | Test | Konst. | Ep. 0 | $n_{\text{Test}}$ |
| Standard (Gen 1) | 15,29 | 29,22 | 21,52 | 28,70 | 1 / 4 | 69 |
| Zufällige Sprünge (Gen 2) | 23,45 | 29,44 | 32,80 | 34,39 | 2 / 5 | 22 |
| Eckengewichtet (Gen 4) | 19,76 | 31,31 | 28,07 | 31,82 | 5 / 9 | 91 |
| Alle (18) | 19,79 | 30,32 | 27,93 | 31,84[a] | 8 / 18 | 72 |

[a] Mittelwert der Sitzungswerte des sequenziellen Splits.

**Tabelle 4.18:** Testfehler pro Bildschirmregion (Abschnitt 4.3.1, $r = 0{,}15$), Mittelwert über die Sitzungen einer Gestaltung; in Klammern die Gesamtzahl der Testframes in der Region. Basismodell `norm_subject`.

| | Zufälliger Split | | | | Sequenzieller Split | | | |
|---|---|---|---|---|---|---|---|---|
| Gestaltung | Alle | Mitte | Ränder | Ecken | Alle | Mitte | Ränder | Ecken |
| Standard (Gen 1) | 15,3 | 17,7 (20) | 13,5 (67) | 16,7 (83) | 21,5 | 11,9 (25) | 20,9 (64) | 28,9 (52) |
| Zufällige Sprünge (Gen 2) | 23,4 | 13,7 (12) | 21,6 (34) | 26,1 (42) | 32,8 | 6,6 (2)[a] | 29,8 (15) | 38,6 (45) |
| Eckengewichtet (Gen 4) | 19,8 | 11,5 (45) | 14,9 (175) | 26,5 (299) | 28,1 | 9,0 (60) | 23,9 (136) | 37,1 (289) |
| Alle (18) | 19,8 | 13,5 (77) | 16,4 (276) | 24,2 (424) | 27,9 | 9,7 (87) | 24,3 (215) | 36,1 (386) |

[a] Nur eine Sitzung mit zwei Frames in der Mitte; nicht interpretierbar.

**Sequenzieller Split.** Ohne Beinahe-Duplikate steigt der Fehler in jeder Gestaltung, im Mittel von 19,8 % auf 27,9 % (Wilcoxon $p = 4 \cdot 10^{-5}$). Das personalisierte Modell ist immer noch besser als die Konstant-Baseline (Wilcoxon $p = 0{,}002$), die Verringerung schrumpft aber von 35 % auf etwa 12 %. In 8 von 18 Sitzungen (02, 05, 08, 23–27) ist die beste Epoche 0: Das Training auf den ersten 70 % der Aufnahme verbessert den Fehler auf dem Validierungsblock nicht, das angegebene Ergebnis ist also das unveränderte Basismodell. Das geschieht vor allem in Gen 4 mit unruhigem Hintergrund (23–27) und in den kurzen Gen-2-Aufnahmen, deren Validierungs- und Testblöcke nur 17–27 Frames enthalten. Der zufällige Split aus Abschnitt 4.4 überschätzt daher den Nutzen der Personalisierung bei schwierigen Sitzungen stark.

**Gestaltungen.** Bei beiden Splits hat das Standardvideo (Gen 1) den niedrigsten und die zufälligen Sprünge (Gen 2) den höchsten Fehler; auch mit dem sequenziellen Split sind die Unterschiede nicht signifikant (Kruskal-Wallis $p = 0{,}06$). Die Rangfolge lässt sich durch die Abdeckung erklären (Tabelle 4.16): Gen 1 deckt in jedem Teil der Aufnahme 97 % des Bildschirms ab, während bei den Sprung-Gestaltungen ein zeitlicher Block nur die in dieser Zeit gezeigten Ziele abdeckt. Die kurzen Gen-2-Aufnahmen leiden am stärksten darunter. Die Gestaltungen sind weiterhin mit Beleuchtung, Hintergrund und Brille vermengt, die Rangfolge ist also eine Tendenz und kein Beweis.

**Regionen.** Die Ecken sind in jeder Gestaltung und bei beiden Splits die schwierigste Region (24 % zufällig, 36 % sequenziell), die Mitte die einfachste (10–14 %). Die eckengewichtete Gestaltung senkt den Eckfehler *nicht*: Gen 4 hat 26,5 % / 37,1 % in den Ecken, verglichen mit 16,7 % / 28,9 % beim Standardvideo. Mehr Ecklabels allein helfen nicht, wenn der Blick in den Ecken schwerer zu schätzen ist (Kopfdrehung, teilweise geschlossene Augen, Kamerageometrie) und wenn die Ecklabels unter schlechteren Bedingungen aufgenommen wurden (Gen-4-Sitzungen 23–27). Die äußersten Ecken sind außerdem die Punkte, an denen die Sigmoid-Ausgabe nahe 0 oder 1 liegen muss.

**Folgerung für die Gestaltung.** Für eine kurze Kalibrierung liefert mit den vorhandenen Daten eine kontinuierliche Zielbewegung, die den ganzen Bildschirm abdeckt, das beste Ergebnis. Sprünge erzeugen Übergangsframes (Gen 2: bis zu 28 % der Labels) und müssen, wenn sie verwendet werden, protokolliert werden, damit diese Frames entfernt werden können. Die Eckengewichtung erhöht den Anteil der Ecklabels wie beabsichtigt, senkt aber nicht den Eckfehler.

## 4.7 Diskussion

**Hauptergebnis.** Auf GazeCapture erreichen alle Konfigurationen der Hyperparameterstudie praktisch den Fehler eines konstanten Schätzers auf denselben Testdaten: Die besten Konfigurationen mit „nur FC“ liegen zwischen 28,32 % und 28,39 %, die Konstant-Baseline bei 28,46 %. Die räumliche Analyse bestätigt, dass sich die Vorhersagen in der Bildschirmmitte konzentrieren. Folglich trennt die Hyperparameterstudie die Konfigurationen vor allem nach ihrer *Stabilität*: Optionen, die das Training weniger stabil machen (Gauß, ReLU, keine Aktivierung, Auftauen von Backbone-Schichten, sensitiver Verlust), sind schlechter, aber keine Option verlässt deutlich die Mittelpunktlösung. Die Entscheidungen in Abschnitt 4.2 beruhen daher auf Stabilität, Konvergenz, Aufwand und methodischen Argumenten, wie in jedem Unterabschnitt angegeben. Einzelne Notebook-Läufe auf `norm_random` (Abbildungen 4.1b und 4.2) erreichten etwa 23,6 % gegenüber 26,8 % für die Konstant-Baseline; ein kleiner Lerneffekt existiert also, ist aber in den Mittelwerten über fünf Läufe nicht sichtbar. Die Basismodelle der Personalisierung bestätigen das: Ihr Fehler auf den persönlichen Sitzungen (30,6 %) entspricht der Konstant-Baseline.

**Mögliche Ursachen.** Das neue Protokoll schließt bereits zwei Erklärungen aus, die für die früheren Läufe plausibel waren: Die Trainingsteilmenge enthält jetzt 179 statt fünf Personen, und die Testmenge wird nicht mehr für die Modellauswahl verwendet; der Kollaps bleibt bestehen. Die wahrscheinlichsten verbleibenden Ursachen sind: (i) Es werden vollständige Kamerabilder verwendet, in denen die Augen nur einen kleinen Teil des Bildes einnehmen, sodass die eingefrorenen ImageNet-Merkmale wenig Blickinformation tragen; (ii) 1000 Frames von 179 Personen sind nur etwa sechs Frames pro Person, zu wenig, um eine personenunabhängige Abbildung zu lernen; das Auftauen weiterer Schichten mit so wenigen Daten macht das Training instabil statt besser (Abschnitt 4.2.3). Gesichts- oder Augenausschnitte als Eingabe und größere Trainingsmengen sind die nächsten zu prüfenden Schritte.

**Personalisierung.** Mit demselben eingefrorenen Backbone senkt ein Kopf, der auf einigen hundert Frames einer Person trainiert wurde, den Fehler gegenüber der Konstant-Baseline um etwa ein Drittel (zufälliger Split), und das in etwa zwei Sekunden auf einer CPU. Die Wahl des Basismodells spielt keine Rolle. Das stützt die zentrale Idee dieser Arbeit, die dateneffiziente Personalisierung eines vortrainierten Modells. Zwei Einschränkungen sind wichtig: (i) Der zufällige Split ist optimistisch; mit einem zeitlichen Split sinkt die Verringerung im Mittel auf etwa 12 % und in schwierigen Sitzungen auf null, während gute Sitzungen den größten Teil des Gewinns behalten und sich eine Kalibrierung sogar auf eine zweite Aufnahme überträgt (47 % Verringerung, Abschnitt 4.5); (ii) die Aufnahmebedingungen (Hintergrund, Beleuchtung, Brille) beeinflussen das Ergebnis stärker als die Gestaltung der Sitzung.

**Folgen für den Code.** Die Evaluationsmetrik wurde korrigiert, Baselines werden auf derselben Teilmenge wie das Modell berechnet, zufällige Teilmengen (`-sample-seed`) und eine personendisjunkte Validierungsmenge (`-val-size`) wurden zu `exp.py` hinzugefügt, und vollständiges Fine-Tuning (`fc_resnet`) kann mit `-include-resnet` einbezogen werden. Die Personalisierungsläufe sind reproduzierbar (feste Seeds, eingefrorene BatchNorm, Merkmals-Cache, gespeicherte Vorhersagen, `personal_exp.py`).

## 4.8 Zusammenfassung der experimentellen Ergebnisse

Tabelle 4.19 fasst alle Experimente zusammen. „Signifikant“ bedeutet $p < 0{,}05$ im jeweils im Abschnitt angegebenen Test. Alle Fehler sind Diagonalfehler (Gleichung (4.1)) in %. MAE und MSE wurden in diesen Experimenten nicht protokolliert; sie werden daher nicht angegeben, und der Diagonalfehler (ein skalierter euklidischer RMSE) ist das einzige Genauigkeitsmaß, das für alle Experimente verfügbar und vergleichbar ist.

**Tabelle 4.19:** Zusammenfassung der experimentellen Ergebnisse und der gewählten Konfiguration. GazeCapture: `norm_subject`, 1000/200 zufällige Frames, fünf Läufe, Konstant-Baseline 28,46 %; Zeit = mittlere Dauer eines Trainingslaufs in Minuten (Kopf auf zwischengespeicherten Merkmalen). Personalisierung: Mittelwert über die Sitzungen, Zeit = ein Sitzungslauf in Sekunden (Abschnitt 4.4.3).

| Experiment | Gewählt | Fehler (%) | Zeit | Ergebnis und Begründung |
|---|---|---|---|---|
| Baselines | — | 28,46 | — | Konstant-Baseline auf denselben Testdaten; Zufalls-Baseline 40,36 (Abschnitt 4.1.6) |
| Zieldarstellung | normiert $[0,1]^2$ | — | — | geräteunabhängig; Metrik wohldefiniert (Abschnitt 4.1.5) |
| Batchgröße | 32 | 28,39 | 0,019 min | kein signifikanter Unterschied (Abschnitt 4.2.1) |
| Lernrate | $10^{-4}$ | 28,37 | 0,020 min | kein signifikanter Unterschied; beste Konvergenz in den Kurven (Abschnitt 4.2.2) |
| Trainierbare Schichten | nur FC | 28,36 | 0,015 min | niedrigster und stabilster Fehler; FC + Layer 4: 9,2 min, voll: 15,4 min (Abschnitt 4.2.3) |
| Aktivierung | Sigmoid | **28,36** | 0,022 min | **signifikant** besser als Gauß, ReLU, Keine (Abschnitt 4.2.4) |
| Verlustfunktion | L1 | **28,32** | 0,011 min | **signifikant** besser als der sensitive Verlust (Abschnitt 4.3.4) |
| Datensatzgröße | 1000/200 | 28,39 | 0,018 min | größere Menge nur wegen anderer Testmenge besser (Abschnitt 4.2.5) |
| Datensplit | `norm_subject` | 28,39 | 0,020 min | einziger Split, der die Generalisierung auf unbekannte Personen misst (Abschnitt 4.2.6) |
| Personalisierung (E1) | nur FC, 18 Sitzungen | **19,79** | 1,9 s | **signifikant** besser als Konstant-Baseline (30,32) und Basismodell (30,57), −35 % (Abschnitt 4.4.2) |
| Basismodell (E0/E1) | `norm_subject` | 19,79 | 1,9 s | gleich wie `norm_random` (19,59; 2,4 s; $p = 0{,}20$); gewählt, weil es keine Information über die Testpersonen enthält |
| Label-Bereinigung (E1b) | WAIT-Frames entfernen | 15,88 | — | vs. 16,72 auf denselben MOVE-Testframes; −0,84, nicht signifikant; WAIT-Frames selbst 37,7 (Abschnitt 4.4.4) |
| Laden der Daten (E2) | sequenziell / zwei Videos | 15,55 / 17,24 | 2–5 s | zufällig 13,85 ist optimistisch; zwei Videos signifikant schlechter als sequenziell; Kalibrierung überträgt sich auf ein neues Video (−47 %) (Abschnitt 4.5) |
| Sitzungsgestaltung (E3) | Standardvideo | 15,29 / 21,52 | — | zufälliger / sequenzieller Split; zufällige Sprünge 23,45 / 32,80, eckengewichtet 19,76 / 28,07; nicht signifikant und vermengt; Eckengewichtung senkt den Eckfehler nicht (Abschnitt 4.6.2) |

# Literaturverzeichnis

[1] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik, and Antonio Torralba. Eye tracking for everyone. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2176–2184, 2016. doi: 10.1109/CVPR.2016.239.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[3] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.